

Implementation Guide for the LDM Compression Model

Based on *High-Resolution Image Synthesis with Latent Diffusion Models* and *Taming Transformers for High-Resolution Image Synthesis*

1 Overview

This document describes a two-stage model for compressing images into a latent space suitable for training diffusion models. The model consists of:

- An autoencoder that compresses images using a convolutional architecture (implemented as a UNet) with either KL-regularization or vector quantization (VQ).
- A diffusion model that operates on the learned latent representations.
- In some cases, an adversarial loss is added to improve perceptual quality.

2 Stage 1: Autoencoder-Based Compression

2.1 Model Structure

Encoder and Decoder:

The encoder E and decoder D are implemented using UNet-like architectures. The UNet structure is advantageous because it retains spatial information at multiple scales via skip connections. In our setting:

- **Input:** An image $\mathbf{x} \in \mathbb{R}^{H \times W \times 3}$.
- **Encoder:** Maps $\mathbf{x}$ to a latent representation

$$\mathbf{z} = E(\mathbf{x}), \quad \mathbf{z} \in \mathbb{R}^{h \times w \times c},$$

where $h = \frac{H}{f}$, $w = \frac{W}{f}$, and f is a downsampling factor (typically $f = 2^m$, with $m \in \mathbb{N}$).

- **Decoder:** Reconstructs the image from the latent code,

$$\tilde{\mathbf{x}} = D(\mathbf{z}) \approx \mathbf{x}.$$

2.2 Latent Regularization Variants

Two primary strategies are used to regularize the latent space:

2.2.1 KL-Regularized (VAE-like) Compression

- **Latent Prior:** Impose that the latent distribution $q(\mathbf{z}|\mathbf{x})$ is close to a standard normal $\mathcal{N}(0, I)$.

- **KL Loss:**

$$\mathcal{L}_{\text{KL}} = D_{\text{KL}}\left(q(\mathbf{z}|\mathbf{x}) \parallel \mathcal{N}(0, I)\right).$$

- **Total Loss:**

$$\mathcal{L}_{\text{total}} = \|\mathbf{x} - \tilde{\mathbf{x}}\|^2 + \beta \mathcal{L}_{\text{KL}},$$

where β is a hyperparameter.

2.2.2 Vector Quantized (VQ) Compression

- **Quantization:** After obtaining the continuous latent $\hat{\mathbf{z}} = E(\mathbf{x})$, each spatial vector $\hat{\mathbf{z}}_{ij} \in \mathbb{R}^c$ is quantized to the nearest codebook vector in $\mathcal{Z} = \{\mathbf{z}_k\}_{k=1}^K$:

$$\mathbf{z}_q(i, j) = q(\hat{\mathbf{z}}_{ij}) = \arg \min_{\mathbf{z}_k \in \mathcal{Z}} \|\hat{\mathbf{z}}_{ij} - \mathbf{z}_k\|.$$

- **Reconstruction:**

$$\tilde{\mathbf{x}} = D(\mathbf{z}_q).$$

- **Loss Terms:**

$$\mathcal{L}_{\text{rec}} = \|\mathbf{x} - \tilde{\mathbf{x}}\|^2, \tag{1}$$

$$\mathcal{L}_{\text{code}} = \|\text{sg}[E(\mathbf{x})] - \mathbf{z}_q\|^2, \tag{2}$$

$$\mathcal{L}_{\text{commit}} = \|\text{sg}[\mathbf{z}_q] - E(\mathbf{x})\|^2. \tag{3}$$

- **Total VQ Loss:**

$$\mathcal{L}_{\text{VQ}} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{code}} + \mathcal{L}_{\text{commit}}.$$

Here, $\text{sg}[\cdot]$ denotes the stop-gradient operator.

2.3 Optional: Adversarial Training

To ensure reconstructions lie on the natural image manifold, a patch-based discriminator D_{adv} can be employed:

$$\mathcal{L}_{\text{GAN}} = \log D_{\text{adv}}(\mathbf{x}) + \log(1 - D_{\text{adv}}(\tilde{\mathbf{x}})).$$

The combined loss for the VQ variant becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{VQ}} + \lambda \mathcal{L}_{\text{GAN}},$$

with λ typically computed adaptively.

3 Stage 2: Diffusion Modeling in Latent Space

Once the autoencoder is trained, the latent representations serve as the basis for the diffusion model.

3.1 Diffusion Model Loss

The diffusion model is implemented as a time-conditional UNet, which is applied in the learned latent space. The training objective is:

$$\mathcal{L}_{\text{DM}} = \mathbb{E}_{\mathbf{x}, \epsilon, t} \left[\|\epsilon - \epsilon_{\theta}(z_t, t)\|^2 \right],$$

where:

- $\mathbf{x}$ is an input image,
- $\epsilon \sim \mathcal{N}(0, 1)$ is the noise,
- z_t is the latent representation at noise level t , and
- ϵ_{θ} is the denoising network (the UNet backbone).

3.2 Reconstruction at Inference

During inference, the diffusion model generates a latent representation $\mathbf{z}$, which is then decoded to produce the final image:

$$\tilde{\mathbf{x}} = D(\mathbf{z}).$$

4 Transformer for Codebook Modeling

In the second paper, a VQGAN is used to learn a discrete codebook of visual tokens, and a transformer is applied to model the global composition of images.

- **Discrete Representation:** The quantized latent $\mathbf{z}_q$ is converted into a sequence of indices:

$$s_{ij} = k \quad \text{if } (\mathbf{z}_q)_{ij} = \mathbf{z}_k.$$

- **Autoregressive Modeling:** The transformer is then trained to model the distribution over the sequence:

$$p(s) = \prod_i p(s_i | s_{<i}),$$

with the negative log-likelihood loss:

$$\mathcal{L}_{\text{Transformer}} = -\log p(s).$$

- **Conditional Generation:** If additional conditioning information (e.g., class labels, segmentation maps) is available, the transformer models:

$$p(s|c) = \prod_i p(s_i | s_{<i}, c).$$

5 Training Procedure Summary

1. **Data Preparation:** Preprocess and normalize images.
2. **Autoencoder Training:**
 - (a) For each image $\mathbf{x}$, compute $\hat{\mathbf{z}} = E(\mathbf{x})$.
 - (b) Apply quantization (if using VQ) to obtain $\mathbf{z}_q$.
 - (c) Reconstruct the image: $\tilde{\mathbf{x}} = D(\mathbf{z})$ or $\tilde{\mathbf{x}} = D(\mathbf{z}_q)$.
 - (d) Compute the reconstruction loss along with either KL or VQ losses (plus optional adversarial loss).
 - (e) Backpropagate and update E , D , and (if applicable) the codebook $\mathcal{Z}$.
3. **Diffusion Model Training:**

- (a) Encode images using the trained encoder E to obtain latent codes.
 - (b) Train the time-conditional UNet diffusion model using $\mathcal{L}_{\text{DM}}$.
4. **Transformer Training (Optional):** For applications requiring discrete codebook modeling, train a transformer to autoregressively model the sequence of indices derived from $\mathbf{z}_q$.

6 Conclusion

This guide presented a step-by-step overview to implement the LDM compression model. The key components include:

- A UNet-based autoencoder that compresses images into a latent space using either KL-regularization or vector quantization.
- Optional adversarial training to improve reconstruction quality.
- A diffusion model (with a UNet backbone) trained in the latent space.
- An autoregressive transformer for modeling the codebook indices if discrete representations are desired.

This modular approach allows efficient high-resolution image synthesis by first compressing the image data and then applying generative diffusion or transformer-based models in the lower-dimensional latent space.